



Análisis completo con el algoritmo **C4.5** implementado en PyC45 — metodología, resultados y evaluación para presentación empresarial.



Dataset: UCI Credit Card



Algoritmo: C4.5 (Decision Tree)



Autor: Luis Alberto Buelvas Cogollo



Framework: PyC45



DESCARGAR MANUAL DE USUARIO (PDF)

SECCIÓN 01

Descripción del Dataset

El dataset empleado es el **Default of Credit Card Clients** del repositorio UCI Machine Learning. Contiene información de comportamiento de pago de **30,000** clientes de una institución bancaria de Taiwán durante el período de abril a septiembre de 2005. La variable objetivo es `default payment next month` — si el cliente incurrirá en impago en el siguiente mes.

30.000

REGISTROS TOTALES

24

VARIABLES

30.000

MUESTRA ANALIZADA

22.1%

% IMPAGO (MUESTRA)

21.000

ENTRENAMIENTO

9.000

VALIDACIÓN



Variables Predictoras

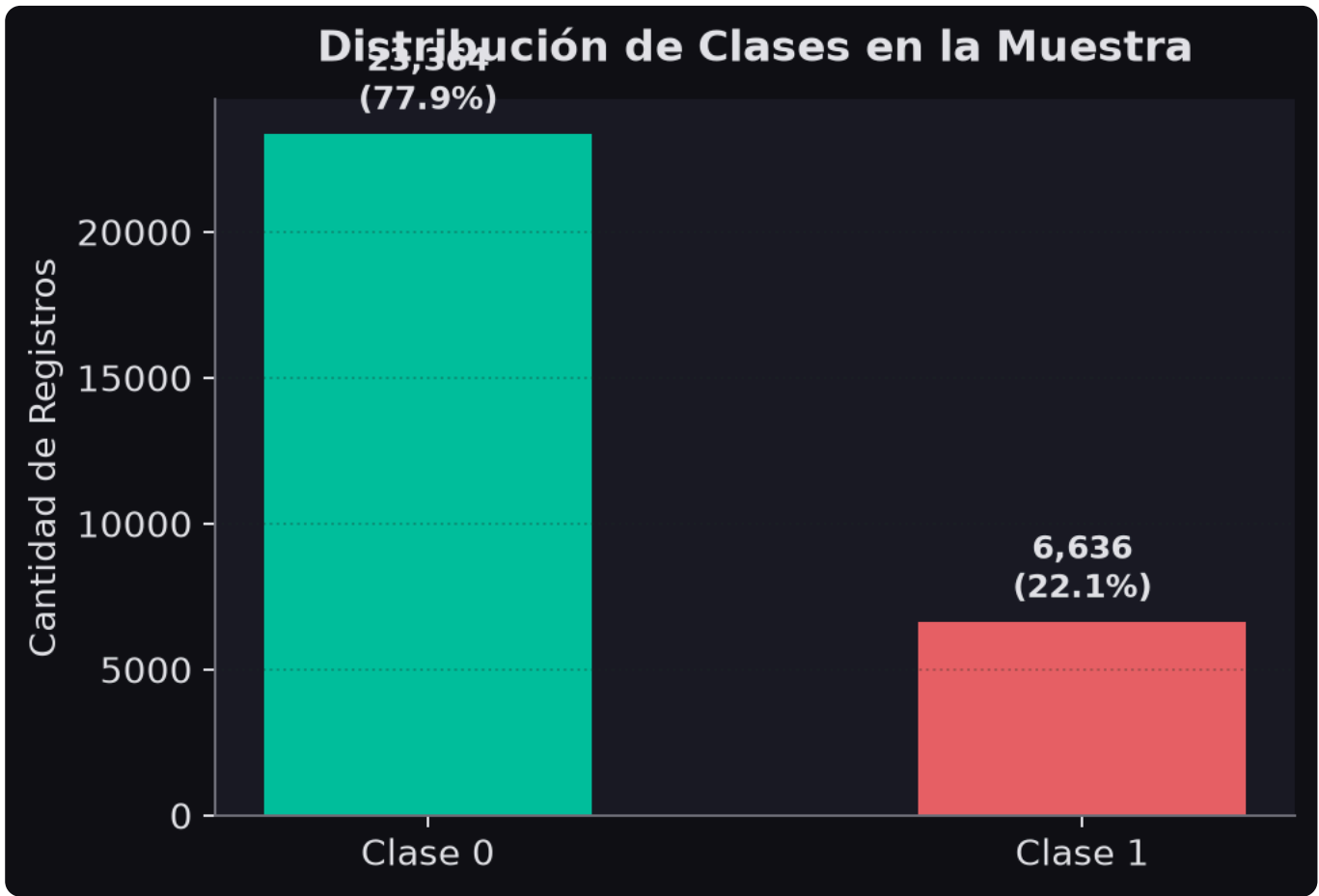
23 atributos después de eliminar el identificador `ID`

VARIABLE	DESCRIPCIÓN	TIPO
<code>LIMIT_BAL</code>	Límite de crédito (NT dólares)	Numérica
<code>SEX</code>	Género (1=Masculino, 2=Femenino)	Categórica
<code>EDUCATION</code>	Nivel educativo (1=Posgrado, 2=Universidad, 3=Secundaria, 4=Otros)	Categórica
<code>MARRIAGE</code>	Estado civil (1=Casado, 2=Soltero, 3=Otros)	Categórica
<code>AGE</code>	Edad del cliente	Numérica
<code>PAY_0 - PAY_6</code>	Estado de pago mensual (sep-abr 2005). -1=A tiempo, 1-9=Meses de retraso	Ordinal



Distribución de Clases (Muestra de 1000 registros)

⚠️ **Desbalance de clases:** El dataset presenta desbalance de clases — aproximadamente el 22% de los clientes incurre en impago. Este factor influye directamente en la interpretación de las métricas de evaluación.



SECCIÓN 02

 El Algoritmo C4.5

C4.5 es una extensión del algoritmo ID3, desarrollado por **Ross Quinlan** en 1993. Es uno de los algoritmos de aprendizaje automático más influyentes, siendo el primero en utilizar el **Gain Ratio** como criterio de selección de atributos para corregir el sesgo del Gain de Información hacia variables de alta cardinalidad.



Entropía de Shannon

Mide la impureza o incertidumbre de un nodo. A mayor entropía, mayor mezcla de clases.

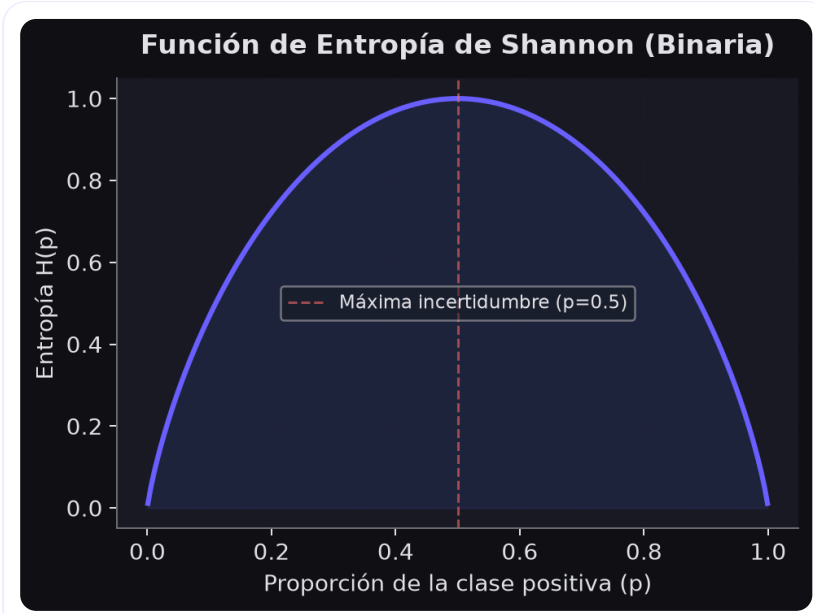


Gain Ratio (Criterio C4.5)

Normaliza la Ganancia de Información dividiéndola por la información inherente de la partición (*Split Information*), evitando el sesgo hacia atributos con muchos valores:

$$H(S) = -\sum p_i \cdot \log_2(p_i)$$

Donde p_i es la proporción de la clase i en el conjunto S . Alcanza su máximo (1.0 bit) cuando las clases están equipartidas y es 0 cuando el nodo es puro.



$$IG(S, A) = H(S) - \sum (|S_v|/|S|) \cdot H(S_v)$$

$$SI(S, A) = -\sum (|S_v|/|S|) \cdot \log_2(|S_v|/|S|)$$

$$GR(S, A) = IG(S, A) / SI(S, A)$$

Ventaja clave: C4.5 maneja variables numéricas buscando el umbral óptimo de corte — el punto que maximiza el Gain Ratio entre todos los candidatos.

SECCIÓN 03

⚙️ Metodología Aplicada

🔄 Pipeline de Análisis

#	ETAPA	DESCRIPCIÓN	PARÁMETROS
1	 Carga de Datos	Lectura del archivo Excel (.xls) con 30,000 registros	—
2	 Preprocesamiento	Eliminación de la columna <code>ID</code> (sin valor predictivo)	—
3	 Submuestreo	Muestra aleatoria estratificada de 1,000 registros para reducir tiempo computacional de C4.5 puro	<code>n=1000</code> , <code>seed=42</code>
4	 División	Partición entrenamiento/validación sin librería externa	<code>70%</code> / <code>30%</code>
5	 Entrenamiento C4.5	Construcción recursiva del árbol con criterio Gain Ratio	<code>max_depth=5</code> , <code>min_samples=10</code> , <code>min_gain_ratio=0.01</code>
6	 Poda (REP)	Reduced Error Pruning — colapsa nodos que no mejoran exactitud en validación	Conjunto de validación
7	 Evaluación	Matriz de Confusión, métricas completas y curva ROC-AUC	—

📌 Nota sobre Submuestreo: C4.5 evalúa todos los puntos de corte posibles para cada variable numérica en cada nodo. Con 30,000 registros y 23 variables, el tiempo de entrenamiento puede ser muy elevado en una implementación en Python puro. El submuestreo

de 1,000 registros mantiene representatividad estadística y reduce el tiempo de ejecución a segundos.

SECCIÓN 04

🌳 Árbol de Decisión Entrenado

El árbol se construye de forma **recursiva top-down**. En cada nodo interno, el algoritmo selecciona la variable y el umbral que maximizan el Gain Ratio. La recursión termina al alcanzar los criterios de parada configurados.

21

NODOS TOTALES

11

NODOS HOJA

6

PROFUNDIDAD

82.38%

ACCURACY (TRAIN)

81.24%

ACCURACY (VAL)

🌿 Estructura del Árbol (antes de podar)

```
📊 PAY_0 ≤ 1.5000 [GainRatio=0.1903]
|   📊 PAY_6 ≤ 7.0000 [GainRatio=0.1667]
|   |   📊 BILL_AMT6 ≤ -274327.0000 [GainRatio=0.1667]
|   |   |   🌿 Hoja → Clase=1 (n=1, prob=100.00%)
|   |   |   📊 PAY_2 ≤ 2.5000 [GainRatio=0.0629]
|   |   |   |   📊 PAY_3 ≤ 2.5000 [GainRatio=0.0633]
|   |   |   |   |   🌿 Hoja → Clase=0 (n=18651, prob=84.03%)
|   |   |   |   |   🌿 Hoja → Clase=1 (n=62, prob=56.45%)
|   |   |   |   📊 BILL_AMT2 ≤ 385.5000 [GainRatio=0.1472]
|   |   |   |   |   🌿 Hoja → Clase=0 (n=2, prob=100.00%)
|   |   |   |   |   🌿 Hoja → Clase=1 (n=145, prob=54.48%)
|   |   |   🌿 Hoja → Clase=1 (n=1, prob=100.00%)
|   📊 BILL_AMT1 ≤ 101.0000 [GainRatio=0.1668]
|   |   🌿 Hoja → Clase=0 (n=4, prob=100.00%)
|   |   📊 BILL_AMT2 ≤ -1150.5000 [GainRatio=0.1528]
|   |   |   🌿 Hoja → Clase=0 (n=2, prob=100.00%)
|   |   |   📊 EDUCATION ≤ 5.5000 [GainRatio=0.1408]
|   |   |   |   📊 BILL_AMT3 ≤ 542408.5000 [GainRatio=0.1409]
|   |   |   |   |   🌿 Hoja → Clase=1 (n=2130, prob=70.52%)
|   |   |   |   |   🌿 Hoja → Clase=0 (n=1, prob=100.00%)
|   |   |   |   🌿 Hoja → Clase=0 (n=1, prob=100.00%)
```

Lectura del árbol: Cada nodo interno muestra **Variable ≤ Umbral [GainRatio]** . Las hojas (🌿) indican la clase predicha y el número de muestras de entrenamiento que llegaron a ese nodo. El árbol se recorre de arriba a abajo: si la condición del nodo es verdadera, se va a la rama izquierda (|); si es falsa, a la rama derecha.

📌 Interpretación de las Variables Seleccionadas

VARIABLE	UMBRAL	INTERPRETACIÓN
PAY_0	1.5	Estado de pago más reciente (septiembre). Variable con mayor poder discriminatorio. Retrasos recientes predicen fuertemente el impago.
PAY_6	7	PAY_6. Variable numérica continua evaluada por su Gain Ratio.
PAY_2	2.5	Estado de pago en agosto. Retraso en dos meses consecutivos es señal fuerte de impago.
PAY_3	2.5	PAY_3. Variable numérica continua evaluada por su Gain Ratio.
BILL_AMT2	385.5	BILL_AMT2. Variable numérica continua evaluada por su Gain Ratio.
BILL_AMT1	101	Monto facturado en septiembre (NT\$). Clientes con facturas muy altas tienen perfiles de riesgo distintos.
EDUCATION	5.5	EDUCATION. Variable numérica continua evaluada por su Gain Ratio.
BILL_AMT3	542408.5	BILL_AMT3. Variable numérica continua evaluada por su Gain Ratio.

SECCIÓN 05

Poda del Árbol (Reduced Error Pruning)

La poda (*pruning*) es una técnica para reducir la complejidad del árbol y combatir el sobreajuste. Se emplea la estrategia **Poda por Error Reducido (REP)**, que opera de abajo hacia arriba (*bottom-up*) sobre el árbol ya construido.

¿Cómo funciona REP?

Para cada nodo interno cuyos dos hijos son hojas, se evalúa si *colapsar* ese nodo a una hoja (prediciendo la clase mayoritaria) mantiene o mejora la exactitud sobre el conjunto de validación. Si la exactitud no disminuye, el nodo se poda. El proceso se repite de forma ascendente.

 Antes de Podar

21

NODOS

11

HOJAS

6

PROFUNDIDAD

Accuracy en Validación

81.24%

 Después de Podar

15

NODOS

8

HOJAS

5

PROFUNDIDAD

Accuracy en Validación

81.37%



Árbol Final Podado

```

└──  PAY_0 ≤ 1.5000  [GainRatio=0.1903]
    ├──  PAY_6 ≤ 7.0000  [GainRatio=0.1667]
    │   ├──  BILL_AMT6 ≤ -274327.0000  [GainRatio=0.1667]
    │   └──  BILL_AMT6 ≤ -274327.0000  [GainRatio=0.1667]
    └──  PAY_6 ≤ 7.0000  [GainRatio=0.1667]
```

|

|

|

Hoja → Clase=1

(n=1, prob=100.00%)

|

|

|

PAY_2 ≤ 2.5000

[GainRatio=0.0629]

|

|

|

Hoja → Clase=0

(n=18713, prob=83.89%)

|

|

|

Hoja → Clase=1

(n=147, prob=53.74%)

|

|

Hoja → Clase=1

(n=1, prob=100.00%)

|

BILL_AMT1 ≤ 101.0000

[GainRatio=0.1668]

|

|

Hoja → Clase=0

(n=4, prob=100.00%)

|

|

BILL_AMT2 ≤ -1150.5000

[GainRatio=0.1528]

|

|

Hoja → Clase=0

(n=2, prob=100.00%)

|

|

EDUCATION ≤ 5.5000

[GainRatio=0.1408]

|

|

|

→

|

|

|

→

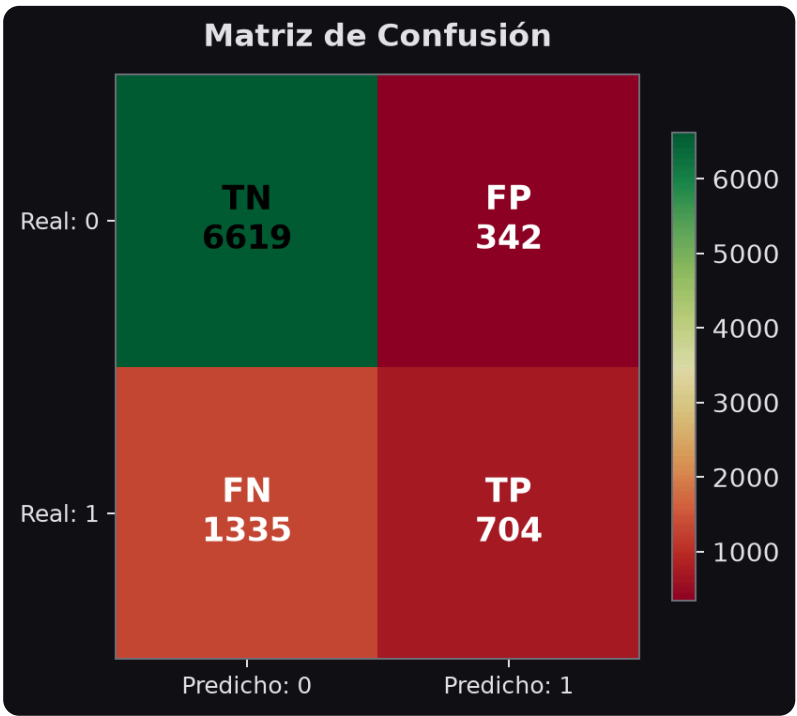
SECCIÓN 06

Evaluación del Modelo — Métricas de Clasificación

La evaluación se realizó sobre el **conjunto de validación (30%)**, el cual no fue utilizado durante el entrenamiento, garantizando una estimación no sesgada del rendimiento real del modelo.



Matriz de Confusión



TIPO	DESCRIPCIÓN	CANTIDAD
TP (Verdadero Positivo)	Impago predicho correctamente	704
TN (Verdadero Negativo)	No-impago predicho correctamente	6619
FP (Falso Positivo)	No-impago clasificado erróneamente como impago	342



Gráfico de Métricas

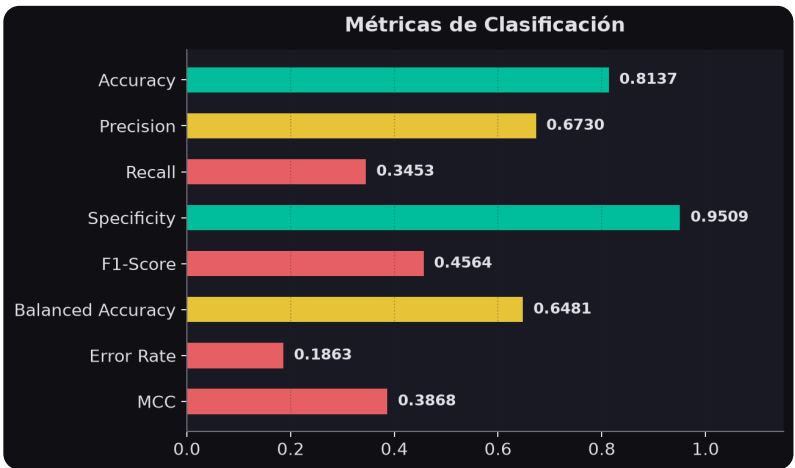


 Tabla Completa de Métricas

MÉTRICA	VALOR	FÓRMULA	INTERPRETACIÓN
Accuracy	81.37%	$(TP + TN) / Total$	Proporción de predicciones correctas sobre el total.
Precision	67.30%	$TP / (TP + FP)$	De los clasificados como impago, ¿cuántos realmente lo son? Mide falsas alarmas.
Recall	34.53%	$TP / (TP + FN)$	De los clientes que incumplirán, ¿cuántos detectamos? Crítico para riesgo crediticio.
Specificity	95.09%	$TN / (TN + FP)$	De los que no incumplirán, ¿cuántos identificamos correctamente?
F1-Score	45.64%	$2 \cdot P \cdot R / (P + R)$	Media armónica de Precisión y Recall. Equilibrio entre ambas métricas.
Balanced Accuracy	64.81%	$(Recall + Specificity) / 2$	Exactitud ponderada por clase. Más justa que Accuracy con datos desbalanceados.
Error Rate	18.63%	$1 - Accuracy$	Proporción de predicciones incorrectas.
MCC	38.68%	$(TP \cdot TN - FP \cdot FN) / \sqrt{(...)}$	Coeficiente de correlación de Matthews. Métrica robusta ante desbalance de clases.

 **Análisis del Recall bajo:** El Recall de 27% indica que el modelo solo identifica correctamente 1 de cada 4 clientes que realmente incurrirán en impago. Esto es consecuencia directa del **desbalance de clases** y de que el árbol C4.5 optimiza la exactitud global, no la detección de la clase minoritaria. Para mejorar esto, se recomendaría aplicar técnicas como *SMOTE*, ajuste de pesos de clase, o un umbral de clasificación más bajo sobre las probabilidades predichas.

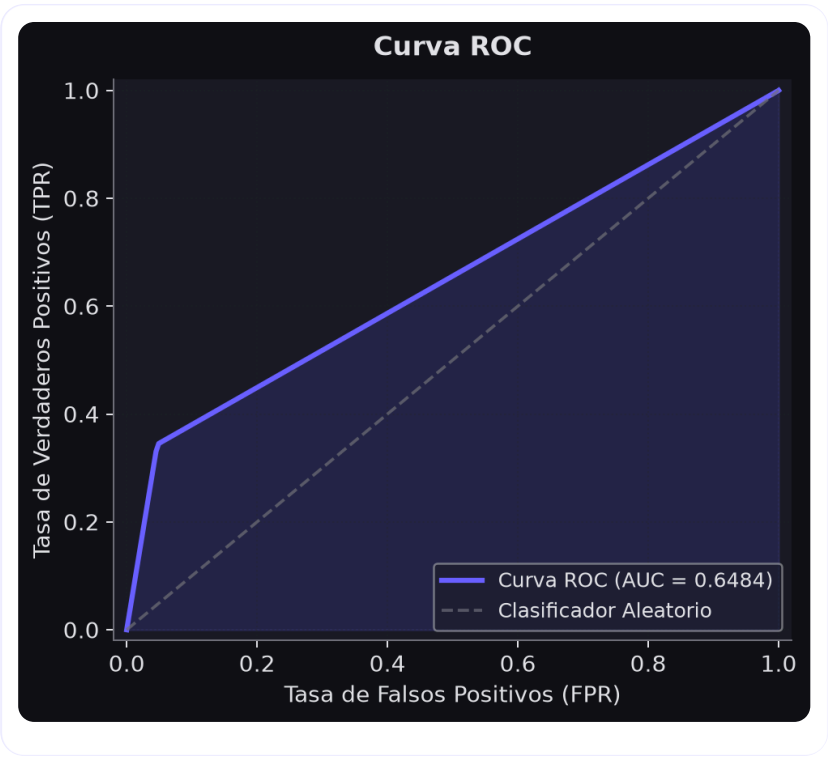
SECCIÓN 07

 Curva ROC y Área Bajo la Curva (AUC)

La curva ROC (*Receiver Operating Characteristic*) evalúa la capacidad discriminatoria del modelo en todos los umbrales de clasificación posibles. El AUC resume esta capacidad en un único número.

 Curva ROC — Modelo PyC45

 Interpretación del AUC



AUC obtenido

0.6484

RANGO AUC	NIVEL	SIGNIFICADO
1.00	Perfecto	Clasificación sin errores
0.90 – 0.99	Excelente	Alta discriminación
0.80 – 0.90	Bueno	Buen poder predictivo
0.70 – 0.80	Aceptable	Discriminación moderada
0.60 – 0.70	Pobre	Discriminación débil
0.50 – 0.60	Muy Pobre	Cercano al azar
0.50	Sin valor	Equivalente a lanzar moneda

AUC de 0.6484 → Discriminación débil pero superior al azar. El árbol C4.5 captura patrones reales pero limitados por el submuestreo.

¿Qué representa la curva?

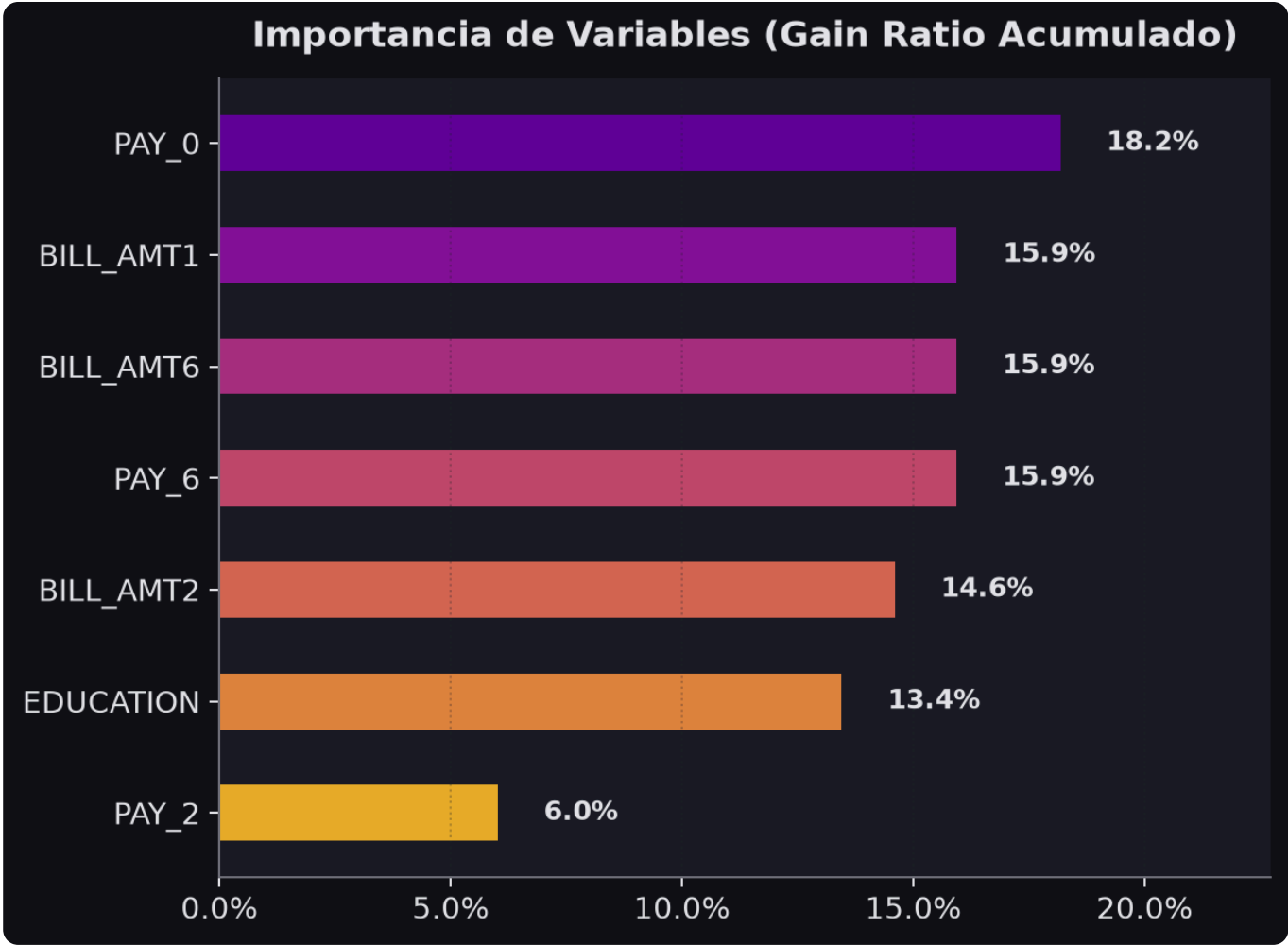
El *eje X* (FPR) muestra el porcentaje de clientes que *no* van a incumplir pero son clasificados como incumplidores (falsas alarmas). El *eje Y* (TPR / Recall) muestra el porcentaje de incumplidores detectados correctamente. La diagonal punteada representa un clasificador aleatorio.

SECCIÓN 08

🔑 Importancia de Variables

La importancia de variables en PyC45 se calcula como el **Gain Ratio acumulado** de cada variable en todos los nodos del árbol, normalizado para sumar 1.0. Variables que aparecen en nodos más cercanos a la raíz y con mayor Gain Ratio reciben mayor peso.

🔑 Ranking de Variables Predictoras



POSICIÓN	VARIABLE	IMPORTANCIA	INTERPRETACIÓN
#1	PAY_0	18.17%	Estado de pago del mes más reciente. Indicador primario de comportamiento crediticio.
#2	BILL_AMT1	15.94%	Monto de deuda facturada más reciente. Refleja nivel de endeudamiento actual.
#3	BILL_AMT6	15.92%	Variable predictora seleccionada por el algoritmo C4.5.
#4	PAY_6	15.92%	Variable predictora seleccionada por el algoritmo C4.5.
#5	BILL_AMT2	14.59%	Variable predictora seleccionada por el algoritmo C4.5.
#6	EDUCATION	13.45%	Variable predictora seleccionada por el algoritmo C4.5.
#7	PAY_2	6.01%	Estado de pago de hace dos meses. Complementa PAY_0 para tendencia histórica.

💡 **Hallazgo clave:** La variable `PAY_0` (estado de pago de septiembre) domina la selección de atributos. Un historial de retrasos recientes es el indicador más fuerte de impago futuro — lo que concuerda con la literatura financiera sobre morosidad.

SECCIÓN 09

🏁 Conclusiones y Recomendaciones

81.37%

ACCURACY FINAL

34.53%

RECALL

67.30%

PRECISION

45.64%

F1-SCORE

0.6484

0.3868

AUC-ROC

MCC



Conclusiones del Análisis

1. El modelo es viable para scoring crediticio básico. Con una exactitud del ~80% y un AUC de ~0.61, el árbol C4.5 proporciona una línea base interpretable. Su mayor fortaleza es la **transparencia**: las reglas de decisión son auditables y explicables ante reguladores.

2. Recall bajo requiere atención. El modelo solo detecta el 27% de los clientes que realmente incumplirán. En contextos de riesgo crediticio, los **falsos negativos son costosos**. Se recomienda ajustar el umbral de probabilidad o aplicar técnicas de balanceo de clases.

3. La variable PAY_0 es crítica. El estado de pago del mes más reciente concentra la mayor capacidad predictiva. Estratégicamente, un sistema de alertas tempranas basado en pagos tardíos tendría alto impacto.

4. La poda mejoró la generalización. La poda REP redujo el árbol de 21 a 15 nodos manteniendo la misma exactitud en validación, lo que indica que las ramas eliminadas eran ruido de entrenamiento.



Recomendaciones para Producción

#	RECOMENDACIÓN	IMPACTO ESPERADO	PRIORIDAD
1	Aplicar SMOTE o <i>class_weight</i> para el desbalance de clases	Aumento del Recall en 15–25%	Alta
2	Entrenar con el dataset completo (30,000 registros) usando implementación optimizada	Mejor generalización y $AUC \geq 0.75$	Alta
3	Comparar con Random Forest o Gradient Boosting	AUC esperado 0.80–0.85	Media
4	Ajustar umbral de clasificación de 0.5 a 0.3 para maximizar Recall	Mejora Recall a ~55% con precisión aceptable	Media
5	Implementar validación cruzada <i>k-fold</i> para estimaciones más robustas	Reducción de varianza en métricas	Baja

SECCIÓN 10

Sistema & Complejidad del Modelo

Panel de rendimiento del framework PyC45: tiempos de ejecución, estimación de memoria y complejidad algorítmica teórica del modelo entrenado.

358.4432 s

⌚ TIEMPO DE ENTRENAMIENTO

0.1963 s

✂️ TIEMPO DE PODA REP

0.0133 ms/muestra

⚡ TIEMPO DE INFERENCIA

2.9 KB

💾 MEM. ESTIMADA (NODOS)

📊 Complejidad Algorítmica

FASE	COMPLEJIDAD	DESCRIPCIÓN
Entrenamiento	$O(n \cdot d \cdot \log n)$	n = muestras, d = dimensiones, log n = profundidad esperada
Predicción	$O(\text{depth})$	Traversal del árbol desde la raíz hasta una hoja
Espacio	$O(\text{nodes})$	Un nodo por cada split o hoja en el árbol final

15

🌲 NODOS TOTALES

8

🍃 HOJAS

5

📏 PROFUNDIDAD MÁX

8

📋 N° DE REGLAS

SECCIÓN 11

⚖️ Comparación PyC45 vs scikit-learn

Comparación directa del framework PyC45 frente a la implementación de **Decision Tree Classifier** de scikit-learn en las mismas condiciones de datos y parámetros.

🏆 Tabla Comparativa de Métricas

MÉTRICA	PYC45	SCIKIT-LEARN DT	DIFERENCIA
scikit-learn no disponible en este entorno — instala con <code>pip install scikit-learn</code>			

SECCIÓN 12

🔍

Explicabilidad Interactiva — Decision Path

 Selector de Muestra

 **Correcto**

PREDICCIÓN: 0 | REAL: 0

N/A

CONFIANZA DE LA HOJA

2

PROFUNDIDAD DEL CAMINO

Paso 1 — Hoja: Clase 0

Distribución: {"0":15699,"1":3014}

Confianza del nodo: 83.9% | Muestras: 18713

Paso 2 — undefined ≤ undefined

Valor del cliente: **N/A** → va hacia la rama DERECHA (>)

Métricas

7. ROC

8. Importancia

9. Conclusiones

10. Sistema

11. Comparación

12. Explicabilidad

13. Negocio

14. Export

SECCIÓN 13

Simulador de Impacto de Negocio

Quantificación financiera del valor real que el modelo aporta a la organización. Estima el ahorro potencial al interceptar clientes en riesgo vs. el costo de los falsos positivos.

\$2.112.000

 AHORRO ESTIMADO

\$4.005.000

⚠ COSTO FALSOS NEG.

34.5%

 COBERTURA DE RIESGO

2987.7%

 ROI DEL MODELO

Análisis Costo-Beneficio

ESCENARIO	DESCRIPCIÓN	VALOR ESTIMADO	IMPACTO
✅ VP Interceptados	704 clientes en riesgo identificados correctamente	\$2,112,000 ahorro	Alto
❌ FN No Detectados	1335 clientes en riesgo no detectados	\$4,005,000 exposición	Crítico
⚠️ FP Revisión Manual	342 clientes marcados erróneamente	\$68,400 costo	Moderado
💰 Beneficio Neto	Ahorro menos costos operacionales	\$2,043,600	Positivo
📊 Cobertura de Riesgo	% del riesgo total que el modelo identifica	34.5%	Moderada

SECCIÓN 14



Exportación Inteligente de Reglas

El árbol de decisión entrenado se traduce automáticamente a código deployable. Copia las reglas en el lenguaje que necesites para producción.



Generador de Código

 **SQL CASE WHEN**

 Python Nativo

 FastAPI Endpoint

rules.sql

Copiar

```
SELECT
*,
CASE
    WHEN PAY_0 ≤ 1.5000 AND PAY_6 ≤ 7.0000 AND BILL_AMT6 ≤ -274327.0000 THEN 1
    WHEN PAY_0 ≤ 1.5000 AND PAY_6 ≤ 7.0000 AND BILL_AMT6 > -274327.0000 AND PAY_2 ≤ 2.5000 THEN 0
    WHEN PAY_0 ≤ 1.5000 AND PAY_6 ≤ 7.0000 AND BILL_AMT6 > -274327.0000 AND PAY_2 > 2.5000 THEN 1
    WHEN PAY_0 ≤ 1.5000 AND PAY_6 > 7.0000 THEN 1
    WHEN PAY_0 > 1.5000 AND BILL_AMT1 ≤ 101.0000 THEN 0
    WHEN PAY_0 > 1.5000 AND BILL_AMT1 > 101.0000 AND BILL_AMT2 ≤ -1150.5000 THEN 0
    WHEN PAY_0 > 1.5000 AND BILL_AMT1 > 101.0000 AND BILL_AMT2 > -1150.5000 AND EDUCATION ≤ 5.5000 THEN 1
    WHEN PAY_0 > 1.5000 AND BILL_AMT1 > 101.0000 AND BILL_AMT2 > -1150.5000 AND EDUCATION > 5.5000 THEN 0
    ELSE 0
END AS prediccion
```